

Process × Virtual Memory

进程身份怎样绑定地址空间，并在 fork 后延迟分化

408 · OS-B02

Mother Interface

Process Identity → Address-Space Association → Shared Mapping → Write Fault → COW Divergence

本 Bridge 连接“谁在执行”和“它看到哪套地址空间”，不重新定义进程生命周期或页表硬件。

Owners 与边界

OS01 Owns process identity、fork/exec 与调度；OS04 Owns address space、PTE、page fault、frame 与 COW 策略。本册 Owns 二者的绑定、共享到分化的交接；硬件 VA→PA 进入 X-B02。

1 共享不是复制

fork 后父子进程通常共享物理页，但各自拥有可区分的地址空间元数据；可写页被暂时标为只读并增加引用关系。读访问仍共享，任一方写入时才触发 fault，OS 分配新页、复制内容、更新写入方映射，随后重试写指令。

2 最小状态轨迹

parent $\xrightarrow{fork}$ parent+child, shared page $\xrightarrow{write}$ fault $\xrightarrow{copy}$ private writable page.

进程切换时，切换的是“当前执行实体 + 地址空间关联”；不能把 context switch 自动等同于复制全部页。

3 最小例子与验证

父进程写入变量前，子进程读取同一变量。若先读后写，读到相同内容；写入方触发 COW 后，父子后续读值可以不同。题目问“是否共享”时必须区分共享虚拟地址、共享物理页、共享文件对象和共享后续写入结果。

Anti-Bridge

fork 的共享映射 ≠ 永久共享可写物理页；COW fault ≠ 普通 TLB miss；进程身份 ≠ 一组固定物理地址。

Compression

当前进程是谁？它绑定哪套 address space？页目前共享什么？哪次写入触发分化？修复后谁的映射被改？